Rust Cheatsheet

FAST • SAFE • CONCURRENT • EXPRESSIVE

A friendly, visual one-pager for the language that loves you back.
Beginner-friendly — but enough depth to keep around forever.

Inside


```rust
fn main() {
    let name = "world";
    println!("Hello, {name}!");
}
```

Designed with care · Generated for leejw51@gmail.com · 2026

Variables & Mutability

```
let x = 5;            // immutable
let mut y = 10;       // mutable
y += 1;
const MAX: u32 = 100; // const
let z: f64 = 3.14;    // typed

// Shadowing (re-declare)
let s = "5";
let s: i32 = s.parse().unwrap();
```

Primitive Types

`i32, u64`	signed / unsigned ints
`f32, f64`	floating point
`bool`	true | false
`char`	Unicode scalar (4 bytes)
`&str`	string slice (borrowed)
`String`	owned, growable string
`(T, U)`	tuple
`[T; N]`	fixed array
`&[T]`	slice (view into array)
`Vec<T>`	growable vector

Strings

```
let s1: &str = "hello";  // slice
let s2 = String::from("hi");
let s3 = s2.clone() + " there";

let n = 7;
let f = format!("n = {n}");

for ch in "rust".chars() {
    println!("{ch}");
}
```

Numbers at a Glance

Bit width →

usize / isize match the pointer width.

```
let a: i32 = 100;
let b = a as f64;       // cast
let c = i64::MAX;
```

Operators

`+ - * / %`	arithmetic
`== != < >`	comparison
`&& || !`	logical
`& | ^ << >>`	bitwise
`= += -=`	assignment
`.. ..=`	ranges (excl / incl)
`?`	propagate error
`& &mut`	borrow / mutable borrow
`*`	dereference

Print & Read

```
println!("{} + {} = {}", 1, 2, 3);
eprintln!("oops: {err}");

use std::io::{self, BufRead};
let mut line = String::new();
io::stdin().read_line(&mut line)?;
let n: i32 = line.trim().parse()?;

// Debug vs Display
println!("{:?}", vec![1, 2, 3]);
println!("{:#?}", point);  // pretty
```

if / else (an expression!)

```
let n = 7;
if n % 2 == 0 {
    println!("even");
} else if n > 0 {
    println!("odd+");
} else {
    println!("neg");
}

// returns a value
let kind = if n > 0 { "pos" } else { "neg" };
```

Loops

```
// loop  -  forever (use break)
let v = loop { break 42; };

// while
while n < 10 { n += 1; }

// for over range
for i in 0..5 { print!("{i} "); }

// for over collection
for x in &vec![1, 2, 3] { ... }

// labels: break out of nested
'outer: for i in 0..5 {
    for j in 0..5 { if j>i { break 'outer; } }
}
```

Functions

```
fn add(a: i32, b: i32) -> i32 {
    a + b   // last expr = return
}

fn greet(name: &str) {
    println!("Hi {name}");
}

// Multiple returns via tuple
fn min_max(v: &[i32]) -> (i32, i32) {
    (*v.iter().min().unwrap(),
     *v.iter().max().unwrap())
}
```

Closures

```
// inferred types
let add = |a, b| a + b;
let n = add(2, 3);

// captures by reference / move
let s = String::from("hi");
let say = move || println!("{s}");
say();

// passed as fn arg
let v: Vec<i32> = (1..=5)
    .map(|x| x * x)
    .collect();
```

Iterator Power Tools

```
let v = vec![1, 2, 3, 4, 5];

let sum: i32 = v.iter().sum();
let max = v.iter().max();

let squares: Vec<_> = v.iter()
    .map(|x| x * x)
    .filter(|x| x % 2 == 0)
    .collect();

// fold = reduce
let p = v.iter().fold(1, |a, x| a * x);

// enumerate / zip / chain
for (i, x) in v.iter().enumerate() {}
```

Match (preview)

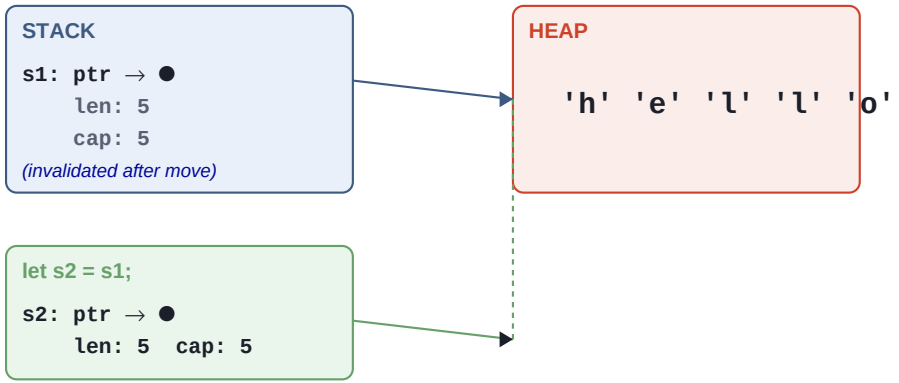
```
let n = 5;
match n {
    0       => println!("zero"),
    1 | 2   => println!("small"),
    3..=9   => println!("single digit"),
    _       => println!("big"),
}

// match returns a value
let label = match n {
    0 => "zero",
    _ if n < 0 => "negative",
    _ => "positive",
};
```

The Three Rules — Visualized

1. Each value has exactly one OWNER.
2. When the owner goes out of scope, the value is DROPPED.
3. You can borrow many times immutably, OR once mutably — never both.

Move semantics — String:



Moving transfers the pointer (cheap). No deep copy. s1 is no longer usable.

Move vs Borrow vs Clone

```
let s1 = String::from("hello");
let s2 = s1;             // move (s1 invalid)

let s3 = s2.clone();     // deep copy

fn print(s: &String)     { println!("{s}"); }
print(&s2);              // borrow (immutable)

fn shout(s: &mut String){ s.push_str("!"); }
let mut m = String::from("hi");
shout(&mut m);           // borrow (mutable)
```

Borrow Checker Rules

- Multiple immutable refs (&T) are fine.
- Only ONE mutable ref (&mut T) at a time.
- Refs must always be valid (no dangling).
- Can't mix &T and &mut T in the same scope.

Copy Types (no move, just copied)

```
// Implement Copy: cheap to duplicate
let a = 5;
let b = a;       // a is still usable

Copy types: integers, floats, bool, char,
            tuples of Copy types, &T (refs).
            NOT String, Vec, Box.
```

Structs

```rust
struct Point { x: f64, y: f64 }

let p = Point { x: 1.0, y: 2.0 };
println!("({}, {})", p.x, p.y);

// tuple struct
struct Color(u8, u8, u8);
let red = Color(255, 0, 0);

// unit struct
struct Marker;

// update syntax
let p2 = Point { x: 3.0, ..p };
```

Methods (impl)

```rust
impl Point {
    // associated fn (no self)
    fn origin() -> Self {
        Self { x: 0.0, y: 0.0 }
    }

    // method (takes &self)
    fn distance(&self, o: &Self) -> f64 {
        ((self.x - o.x).powi(2)
         + (self.y - o.y).powi(2)).sqrt()
    }

    // mutates
    fn shift(&mut self, dx: f64) {
        self.x += dx;
    }
}
```

Enums

```rust
enum Shape {
    Circle(f64),
    Square(f64),
    Rect { w: f64, h: f64 },
    Empty,
}

impl Shape {
    fn area(&self) -> f64 {
        match self {
            Shape::Circle(r) => 3.14*r*r,
            Shape::Square(s) => s*s,
            Shape::Rect{w,h} => w*h,
            Shape::Empty     => 0.0,
        }
    }
}
```

Pattern Matching Magic

```rust
match value {
    Some(0)            => "zero",
    Some(n) if n > 0   => "positive",
    Some(_)            => "negative",
    None               => "missing",
}

// Destructure structs
let Point { x, y } = p;

// Destructure tuples
let (a, _, c) = (1, 2, 3);

// if let / while let
if let Some(v) = maybe { println!("{v}"); }
```

Option<T> & Result<T, E>

```rust
let some_val: Option<i32> = Some(7);
let none_val: Option<i32> = None;

let parsed: Result<i32, _> = "42".parse();

// helpers
let n = some_val.unwrap_or(0);
let m = some_val.map(|x| x + 1);
let ok = parsed.is_ok();

// pattern
match parsed {
    Ok(n) => println!("got {n}"),
    Err(e) => println!("err: {e}"),
}
```

#[derive(...)] Cheats

`Debug`	{:?} printing
`Clone`	explicit deep copy
`Copy`	implicit bitwise copy
`PartialEq`	== and !=
`Eq`	total equality
`Hash`	use as HashMap key
`Default`	T::default()
`Ord`	sortable

#[derive(Debug, Clone, PartialEq)]

Error Handling

```rust
use std::fs;
use std::num::ParseIntError;

// ? propagates errors
fn read_num() -> Result<i32, Box<dyn
                        std::error::Error>> {
    let s = fs::read_to_string("n.txt")?;
    let n: i32 = s.trim().parse()?;
    Ok(n)
}

// panic for unrecoverable
panic!("the unthinkable happened");
```

Vec<T>

```rust
let mut v: Vec<i32> = Vec::new();
v.push(1); v.push(2); v.push(3);

let v = vec![1, 2, 3, 4];
let first = v[0];          // panics if OOB
let safe = v.get(99);      // Option<&T>

v.iter().sum::<i32>();
v.iter().filter(|x| **x > 1);

v.sort();
v.dedup();
v.reverse();
```

HashMap<K, V>

```rust
use std::collections::HashMap;

let mut m = HashMap::new();
m.insert("apples", 3);
m.insert("pears",  5);

// access
let n = m.get("apples");      // Option<&i32>

// upsert
*m.entry("apples").or_insert(0) += 1;

for (k, v) in &m {
    println!("{k}: {v}");
}
```

Slices

```rust
let v = vec![1, 2, 3, 4, 5];
let s = &v[1..4];   // &[2, 3, 4]
let all = &v[..];

fn sum(xs: &[i32]) -> i32 {
    xs.iter().sum()
}

// strings
let g = String::from("greetings");
let hi = &g[0..5];   // "greet"
```

Conversions

`as`	primitive cast
`.to_string()`	anything Display → String
`.parse()`	&str → number / etc
`From / Into`	explicit conversion
`TryFrom`	fallible conversion
`.clone()`	deep copy (heap data)
`.to_owned()`	borrowed → owned
`&v[..]`	Vec → slice
`.iter()`	borrowing iterator
`.into_iter()`	owning iterator

Smart Pointers

```rust
// Box<T>   - heap allocation
let b = Box::new(42);

// Rc<T>    - shared ownership (single-thread)
use std::rc::Rc;
let a = Rc::new(vec![1, 2]);
let b = Rc::clone(&a);

// Arc<T>   - shared ownership (threads)
// RefCell - interior mutability
// Mutex   - mutex-guarded mutability
```

Traits

```
trait Greet {
    fn hi(&self) -> String;

    // default method
    fn shout(&self) -> String {
        self.hi().to_uppercase()
    }
}

struct Dog;
impl Greet for Dog {
    fn hi(&self) -> String {
        "woof".into()
    }
}
```

Generics

```
fn largest<T: PartialOrd>(list: &[T]) -> &T {
    let mut best = &list[0];
    for x in list {
        if x > best { best = x; }
    }
    best
}

struct Pair<T> { a: T, b: T }

impl<T: Copy + std::ops::Add<Output=T>>
    Pair<T>
{
    fn sum(&self) -> T { self.a + self.b }
}
```

Trait Objects (dyn)

```
// static dispatch (monomorphized)
fn print_all<T: Display>(xs: &[T]) {
    for x in xs { println!("{x}"); }
}

// dynamic dispatch (vtable, runtime)
fn render(items: &[Box<dyn Greet>]) {
    for it in items {
        println!("{}", it.hi());
    }
}

// impl Trait return
fn make_greet() -> impl Greet { Dog }
```

Lifetimes

```
// 'a means: lives at least as long as 'a
fn longest<'a>(
    a: &'a str, b: &'a str
) -> &'a str {
    if a.len() > b.len() { a } else { b }
}

struct Excerpt<'a> {
    part: &'a str,
}

// 'static = entire program duration
let s: &'static str = "baked in";
```

Famous Traits

Display	user-facing {}
Debug	developer {:?}
Clone	.clone() copy
Copy	marker: implicit copy
Default	T::default()
Iterator	.next() → Option<Item>
From/Into	T::from(x), x.into()
AsRef	cheap reference conversion
Drop	custom destructor
Send/Sync	thread-safety markers

Modules & Paths

```
// src/lib.rs or src/main.rs
mod garden;             // load garden.rs
mod garden {            // inline mod
    pub fn plant() { }
}

use std::collections::HashMap;
use crate::garden::plant;
use self::sub::*;

pub fn api() {}        // visible outside
pub(crate) fn x() {}   // crate-only
```

Threads

```rust
use std::thread;

let h = thread::spawn(|| {
    for i in 1..5 {
        println!("thread {i}");
    }
});

// move captures
let v = vec![1, 2, 3];
let h = thread::spawn(move || {
    println!("{v:?}");
});

h.join().unwrap();   // wait
```

Channels

```rust
use std::sync::mpsc;
use std::thread;

let (tx, rx) = mpsc::channel();

thread::spawn(move || {
    tx.send("hi").unwrap();
});

let msg = rx.recv().unwrap();

// stream of msgs
for m in rx { println!("got {m}"); }
```

Async / Await

```rust
// requires runtime (tokio, async-std)
async fn fetch(url: &str) -> String {
    // ... await some I/O
    String::from("ok")
}

#[tokio::main]
async fn main() {
    let body = fetch("...").await;
    println!("{body}");
}

// run many concurrently
let (a, b) = tokio::join!(
    fetch("a"), fetch("b")
);
```

Cargo Commands

`cargo new`	create new project
`cargo init`	init in current dir
`cargo build`	compile (debug)
`cargo build --release`	optimized build
`cargo run`	build + run
`cargo test`	run tests
`cargo check`	type-check, no codegen
`cargo fmt`	auto-format
`cargo clippy`	lint
`cargo doc --open`	build & view docs
`cargo add x`	add dependency

Cargo.toml

```toml
[package]
name = "hello"
version = "0.1.0"
edition = "2021"

[dependencies]
serde = { version = "1",
          features = ["derive"] }
tokio = { version = "1",
          features = ["full"] }

[dev-dependencies]
criterion = "0.5"
```

Tests & Macros

```rust
#[test]
fn it_adds() {
    assert_eq!(2 + 2, 4);
}

#[cfg(test)]
mod tests {
    use super::*;
    #[test] fn works() { ... }
}

// handy macros
println! eprintln! format! write!
vec! assert! assert_eq! dbg!
todo!() unimplemented!() panic!()
```

Compile errors are friends — they catch bugs before your users do.

Read them slowly. Most include the exact fix.